



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА У
НОВОМ САДУ



Немања Шимшић

Систем за препоруку тактике над игром *Heroes of Might and Magic III*

ЗАВРШНИ РАД

Први степен, основне академске студије

Нови Сад, 2025.



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА

Редни број, РБР :			
Идентификациони број, ИБР :			
Тип документације, ТД :	Монографска документација		
Тип записа, ТЗ :	Текстуални штампани документ		
Врста рада, ВР :	Дипломски рад		
Аутор, АУ :	Немања Шимшић		
Ментор, МН :	доц. Др Синиша Николић, ФТН Нови Сад		
Наслов рада, НР :	Систем за препоруку тактике над игром <i>Heroes of Might and Magic III</i>		
Језик публикације, ЈП :	Српски		
Језик извода, ЈИ :	Српски/Енглески		
Земља публиковања, ЗП :	Србија		
Уже географско подручје, УГП :	Војводина		
Година, ГО :	2025		
Издавач, ИЗ :	Ауторски репринт		
Место и адреса, МА :	Факултет техничких наука, Трг Доситеја Обрадовића 6, Нови Сад		
Физички опис рада, ФО : (поглавља/страна/ цитата/табела/слика/графика/прилога)	бр. Поглавља 7/страница 43/цитата 36/табела 0/слика 29		
Научна област, НО :	Електротехничко и рачунарско инжењерство		
Научна дисциплина, НД :	Системи базирани на знању		
Предметна одредница/Кључне речи, ПО :	системи базирани на правилима, анализа упитника, представа доменског знања		
УДК			
Чува се, ЧУ :	Библиотека ФТН, Трг Доситеја Обрадовића 6, Нови Сад		
Важна напомена, ВН :			
Извод, ИЗ :	У раду је описан систем за препоруку тактике над игром <i>Heroes of Might and Magic III</i> . Извршен је опис спецификације и имплементација система. Приказано је коришћење апликације од стране нерегистрованих корисника и регистрованих корисника.		
Датум прихватања теме, ДП :			
Датум одбране, ДО :			
Чланови комисије, КО :	Председник:	др Јелена Сливка, проф., ФТН Нови Сад	
	Члан:	др Игор Дејановић, проф., ФТН Нови Сад	
	Члан:		Потпис ментора
	Члан, ментор:	др Синиша Николић, доц., ФТН Нови Сад	



UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES
21000 NOVI SAD, Trg Dositeja Obradovića 6

KEY WORDS DOCUMENTATION

Accession number, ANO :	
Identification number, INO :	
Document type, DT :	Monographic publication
Type of record, TR :	Textual material
Contents code, CC :	Bachelor Thesis
Author, AU :	Nemanja Šimšić
Mentor, MN :	Siniša Nikolic, PhD
Title, TI :	System for recommending tactics for the game <i>Heroes of Might and Magic III</i>
Language of text, LT :	Serbian
Language of abstract, LA :	Serbian/English
Country of publication, CP :	Serbia
Locality of publication, LP :	Vojvodina
Publication year, PY :	2025
Publisher, PB :	Author's reprint
Publication place, PP :	Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	no. of chapters 7/pages 43/quotes 36/tables 0/pictures 29
Scientific field, SF :	Electrical and computer engineering
Scientific discipline, SD :	Knowledge based systems
Subject/Key words, S/KW :	Rule based systems, questionnaire analysis, representation of domain knowledge
UC	
Holding data, HD :	Library of Faculty of Technical Sciences, Trg Dositeja Obradovića 6, Novi Sad
Note, N :	
Abstract, AB :	The paper describes a system for recommending tactics for the game <i>Heroes of Might and Magic III</i> . The specification and implementation of the system are presented. The usage of the application by unregistered users and registered users is demonstrated.
Accepted by the Scientific Board on, ASB :	
Defended on, DE :	
Defended Board, DB :	President: Jelena Slivka, PhD, prof., FTN Novi Sad
	Member: Igor Dejanović, PhD, prof., FTN Novi Sad
	Member:
Member, Mentor:	Siniša Nikolić, PhD, assist. prof., FTN Novi Sad
	Mentor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА		Број:
	21000 НОВИ САД, Трг Доситеја Обрадовића 6		
	ЗАДАТАК ЗА ЗАВРШНИ РАД		Датум:

Студијски програм:	Софтверско инжењерство и информационе технологије		
Студент:	Немања Шимшић	Број индекса:	SV68-2020
Степен и врста студија:	Основне академске студије		
Област:	Електротехничко и рачунарско инжењерство		
Ментор:	др. Синиша Николић		
НА ОСНОВУ ПОДНЕТЕ ПРИЈАВЕ, ПРИЛОЖЕНЕ ДОКУМЕНТАЦИЈЕ И ОДРЕДБИ СТАТУТА ФАКУЛТЕТА ИЗДАЈЕ СЕ ЗАДАТАК ЗА ЗАВРШНИ РАД, СА СЛЕДЕЋИМ ЕЛЕМЕНТИМА: - проблем – тема рада; - начин решавања проблема и начин практичне провере резултата рада, ако је таква провера неопходна;			

НАСЛОВ ЗАВРШНОГ РАДА:

Систем за препоруку тактике над игром <i>Heroes of Might and Magic III</i>
--

ТЕКСТ ЗАДАТКА:

Описати систем за препоруку тактике над игром <i>Heroes of Might and Magic III</i> . Специфицирати и описати архитектуру система. Анализирати и дефинисати правила за препоруку најбољег потеза на основу позиције трупа, доступних магија и јачине трупа, као и правила за одређивање предности у игри. Документовати решење.
--

Руководилац студијског програма:	Ментор рада:

Примерак за: О - Студента; О - Ментора
--

Садржај

1	Увод.....	1
2.	Стање у области.....	3
2.1.	Увод	3
2.2.	Научна област	3
2.3.	Heroes III Assist.....	4
2.4.	Heroes III Tools	4
2.5.	Heroes III Damage Calculator	5
2.6.	FreeHeroes Engine – Battle Emulator	5
2.7.	Водичи за игру.....	6
2.8.	Пресек стања	7
3.	Коришћене технике и технологије.....	9
3.1.	React	9
3.2.	Spring Boot	10
3.3.	Drools.....	11
4.	Спецификација система.....	13
4.1.	Дијаграм случајева коришћења	13
4.2.	Дијаграм класа.....	14
4.3.	Дијаграм секвенце.....	15
5.	Имплементација система.....	19
5.1.	Клијентска апликација.....	19
5.2.	Серверска апликација	21
5.2.1.	Model компонента	21
5.2.2.	Service компонента	24
5.2.3.	Kjar компонента.....	26
6.	Приказ имплементираног система.....	35
6.1.	Register страница.....	35
6.2.	Login страница	35
6.3.	Strategy страница.....	36
7.	Закључак	39
8.	Литература	41
9.	Биографија	43

1 Увод

Heroes of Might and Magic је серијал стратешких видео игара чије је прво издање објављено 1995. године. Од тада је укупно објављено седам игара у серијалу, док је осмо издање (*Heroes of Might and Magic: Olden Era*) најављено за 2025. годину. Међу њима се поготово истиче трећи наставак серијала, *Heroes of Might and Magic III: The Restoration of Erathia* (HoMM3), који је стекао култни статус међу играчима серијала из више разлога. Тај статус је стечен комбинацијом усавршавања самог серијала, док је за популарност игре кључну улогу играла појава пиратерије, локализација на руски и пољски језик, самим тиме и продор на тржишту источне Европе [1].

Један од најбитнијих аспеката HoMM игара јесте *turn-based combat*, где играчи наизменично изводе акције са својим трупима на терену. Сваки играч контролише војске које се састоје из различитих типова јединица, а свака јединица има свој тип, јачину, брзину и домет. Играч планира сваки потез, имајући у виду позицију сваке трупе, редослед напада, доступне магије и предности противничких јединица. Систем борбе у HoMM играма захтева време и праксу како би играч овладао свим доступним тактикама. За усавршавање у систему борби, неопходно је разумети интеракцију различитих типова јединица, коришћење магија у специфичним ситуацијама, као и позиционирање војске на терену [2].

Ова сложеност представља изазов за нове играче који први пут улазе у борбу, као и за играче који желе да се усаврше у игри и развију ефикасне стратегије. Решење овог проблема лежи у развоју система који може да процени стање на терену, препоручи најбоље потезе и помогне играчу у доношењу најјефективнијих одлука.

Тема рада јесте развој система базираног на правилима који ће пружати препоруке за најоптималније потезе током борбе и извршавати процену предности једног од играча. Систем користи податке са битке попут бројности јединица, њихове карактеристике, доступне магије и позиционираност на терену за давање препорука. Циљ рада је приказ да коришћењем система базираних на знању (SBNZ) можемо унапредити процес доношења одлука у стратешким играма попут HoMM3, као и имплементација механизма који омогућава анализу тренутне ситуације у борби, израчунавање *fight point*-ова који одређују ко има предност међу играчима, као и генерисање препорука које играчима могу помоћи у току партије. Систем се ослања на концепт SBNZ-а, где се база знања састоји од продукционих правила која описују понашање јединица у борби.

У другом поглављу се анализира тренутно стање у области, где се даје увид у слична решења. У трећем поглављу се уводе технологије коришћене за имплементацију система. У четвртном поглављу је дефинисана спецификација самог система, као и модел система. У петом поглављу се дискутује о имплементацији целог система, док се у шестом поглављу демонстрира коришћење финалног система. У последњем поглављу су представљене закључне одреднице и потенцијални будући правци развоја апликације.

2. Стање у области

2.1. Увод

Heroes of Might and Magic III (HoMM3) представља једну од најпопуларнијих стратешких игара свих времена. Свака партија захтева од играча пажљиво управљање ресурсима, кретање хероја, планирање експедиција и тактичко позиционирање јединица током борби. Због сложености игре и великог броја променљивих које утичу на исход, појавила се потреба за алатима који пружају додатну подршку у анализи сценарија и предвиђања резултата борби. Те алатке најчешће омогућавају калкулацију штете између јединица, симулација борби, процена релативне снаге армија и визуелизација кретања хероја и јединица.

2.2. Научна област

У последњих неколико деценија, развој видео игара и растућа комплексност њихових механика довеле су до појаве различитих система који омогућавају анализу и оптимизацију начина играња. Један од приступа који се показао корисним у овом контексту јесу системи базирани на знању, који омогућавају доношење одлука на основу претходно представљеног искуства и стратегија искусних играча. Уместо да се одлуке заснивају на интуицији или експериментисању, овакви системи користе унапред дефинисана правила и логичке релације како би предложили најбољи ток игре у одређеној ситуацији.

Рад [3] представља детаљан преглед примене *rule-based* система у развоју видео игара. У овом поглављу аутори описују како се *if-then* правила користе за управљање понашањем непријатеља, доношење одлука и прелазак између различитих стања у игри. Анализа показује да *rule-based* приступ омогућава креирање предвидљивих и контролисаних реакција унутар сложених сценарија игре, што олакшава тестирање и одржавање логике. Иако овај приступ има ограничења у погледу адаптивности, рад наглашава његову вредност као едукативна и практична метода за моделирање понашања у интерактивним окружењима.

Рад [4] описује платформу за генерално играње игара која омогућава формалну репрезентацију правила кроз концепт *ludemes*. У овом систему правила игре се моделују као апстрактне јединице, што омогућава анализу и симулацију различитих стратегија без потребе за модификацијом самих игара. Примена оваквог приступа, *Ludii* може да служи као алат за испитивање оптималних потеза, разумевање сложенијих интеракција унутар игре и подршку при креирању тактичких препорука. Иако систем није намењен директном аутоматском прелазу игара, систем представља ресурс за формализовање знања о механикама и стратегијама.

Рад [5] представља преглед примене *rule-based* система у шаховским програмима попут *Stockfish*. У овим системима правила се користе за процену позиција, избор оптималних потеза и анализу свих могућих варијанти у игри, укључујући комбинацију фигура, контролу центра и потенцијалне тактичке преокрете. Анализа показује да *rule-based* приступ омогућава стварање предвидљивих и прецизних стратегија које се могу систематски евалуирати и тестирати, што пружа људским играчима изазов на високом нивоу и дубље разумевање шаховских позиција. Њихова способност да интегришу правила за процену позиција и доношење одлука истиче практичну вредност *rule-based* приступа у стратешким и тактичким играма.

Рад [6] истражује примену еволуцију у контексту *rule-based* система, са посебним освртом на механизме индукције правила и њихову примену у динамичним окружењима. У овом раду аутори описују како се код током времена шири и адаптира, са фокусом на чињеницу да додавање нових правила омогућава прецизније доношење одлука у сложеним сценаријима. Анализа показује да еволутивни приступ омогућава системима да се прилагођавају и уче на основу искуства, али истовремено захтева механизме за контролу растућег кода како би систем остао ефикасан и лак за одржавање. Рад наглашава вредност *rule-based* приступа као алата који комбинује предвидљивост и контролисану логику са могућношћу адаптације и оптимизације стратегија у интерактивним окружењима.

У наставку су описане и анализиране неке од апликација које се баве анализом сценарија у игри, *Heroes III Assist*, *Heroes III Tools*, *Heroes III Damage Calculator*, *FreeHeroes Engine*, као и водичи за игру.

2.3. Heroes III Assist

Heroes III Assist [7] је самосталан алат који пружа многе функционалности везане за *HoMM3*. Садржи једну од највећих енциклопедија за игру *HoMM3*, и са тиме садржи све јавно доступне информације о магијама, херојима, терену као и јединицама, чиме је омогућен брз приступ информацијама потребним за планирање стратегије. У оквиру саме енциклопедије је могуће додати одређене параметре које чине трупе и магије јачим или слабијим, и у односу на те параметре се мењају представљене вредности, попут јачина трупе или цена магије. Поред тога, омогућава калкулацију штете између јединица, чиме омогућава процену колико ће јединица нанети штету другој током различитих сценарија, укључујући и ефекте посебних способности и магија. *Heroes III Assist* подржава само међусобну борбе између две јединице и тренутно нема подржану опцију за симулацију целе битке, нити систем препоруке следећег најбољег потеза. Екран за процену штете у битки се може видети на слици [Слика 2.1](#). Допунски модули омогућавају прорачуне кретања хероја и потребних ресурса. Алатка по себи је интуитивна и лака за коришћење, и такође садржи податке из неофицијалне експанзије, *Horn of the Abyss*.



Слика 2.1 - *Heroes III Assist*, екран за процену штете

2.4. Heroes III Tools

Heroes III Tools [8] представља веб решење које пружа сличне функционалности попут *Heroes III Assist*-у. Веб алат се састоји из три главна модула: калкулатор штете, калкулатор магије и енциклопедија јединица. Калкулатор штете, сличан калкулатору из *Heroes III Assist*, омогућава процену штете између две трупе, уз додатне параметре који мењају резултат битке, што се може видети на слици [Слика 2.2](#), као и калкулатор магије који кориснику израчунава колико штете наноси одређена магија на одређен тип јединице. Алатка тренутно не подржава променљиве вредности за количину штете нанете противнику, с обзиром да је количина штете увек насумично одабрана између две цифре, већ користи фиксну цифру за количину штете коју јединица наноси. Поред тога пружа информације о свим доступним јединицама, попут напада, снаге, брзине, цене појединачне трупе и одбране.



Слика 2.2 – Heroes III Tools, екран за процену штете

2.5. Heroes III Damage Calculator

Heroes III Damage Calculator [9] је веб алатка креирана од стране Душана Милосављевића, која се специфично фокусира на израчунавање резултата битке између две јединице. Калкулатор штете пружа додатне опције попут подешавања нивоа, напада и одбране хероја који управља јединицама, као и одабир атрибута које јединице могу да имају приликом борбе. Алатка такође подржава променљиве вредности за напад, и приказује просечну штету коју јединица може нанети, као и најмањи и највећи број трупа које може твој напад да убије, што се може видети на слици [Слика 2.3](#). Такође, за разлику од осталих алата, *Heroes III Assist* пружа инструкције о томе како се алатка користи, што доста олакшава коришћење алатке за нове играче.



Слика 2.3 – Heroes III Damage Calculator, екран за процену штете

2.6. FreeHeroes Engine – Battle Emulator

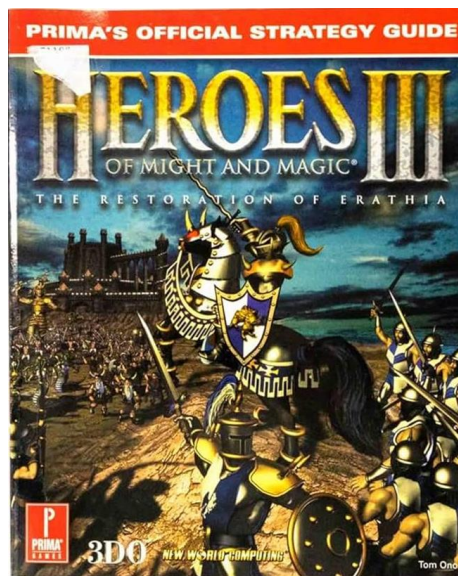
Такође са фокусом на битке, *FreeHeroes Engine – Battle Emulator* [10] омогућава емулацију борби без потребе за главном игром. Покреће се као засебна апликација, и омогућава играчима да тестирају различите

сценарије борби у контролисаном окружењу. Алат пружа детаљан увид у редослед потеза јединица, ефекте магија и посебних способности, као и калкулацију потенцијалне штете. На овај начин играчи могу да анализирају сложене сукобе, испробају различите комбинације јединица и стратегија, и донесу информисане одлуке пре него што примене стратегију у стварној партији. Алат тренутно само симулира праву битку, са потребним уносом од корисника, без опција за препоруку најбољег доступног потеза.

2.7. Водичи за игру

С обзиром на чињеницу да је *HoMM3* изашао 1999. године, у том периоду нису биле толико заступљене алатке за симулацију и калкулацију битки као данас. У то време су се играчи највише ослањали на штампане и електронске водиче за игру, који су пружали детаљне информације о јединицама, чаролијама, херојима и стратегијама за различите сценарије.

Официјални водич објављен од стране Том Оно-а, *Heroes of Might and Magic III, Prima's Official Strategy Guide* [11], је званичан водич за игру и пружа свеобухватне информације о механикама, јединицама, херојима и магијама. Поред основних статистика и описа, водич садржи и детаљне тактике за различите типове сценарија, као и препоруке за оптимално коришћење ресурса. Већи део водича је посвећен анализи битака, али је све то представљено у текстуалној форми, што би значило да играчи морају сами да интерпретирају податке и врше калкулације.



Слика 2.4 - *Heroes III Prima's Official Strategy Guide*

Поред званичног водича, постојали су и неофицијални приручници, као што је *GameStop Unofficial Game Guide to Heroes of Might and Magic III* [12], који су имали другачији приступ. За разлику од детаљног и структурираног формата званичног водича, неофицијални приручници су често били усмерени на практичне савете и брзе трикове за напредовање у игри. Иако су били приступачнији почетницима, овакви водичи су често жртвовали прецизност и потпуност информација, што их је чинило бескорисним за стручније играче.

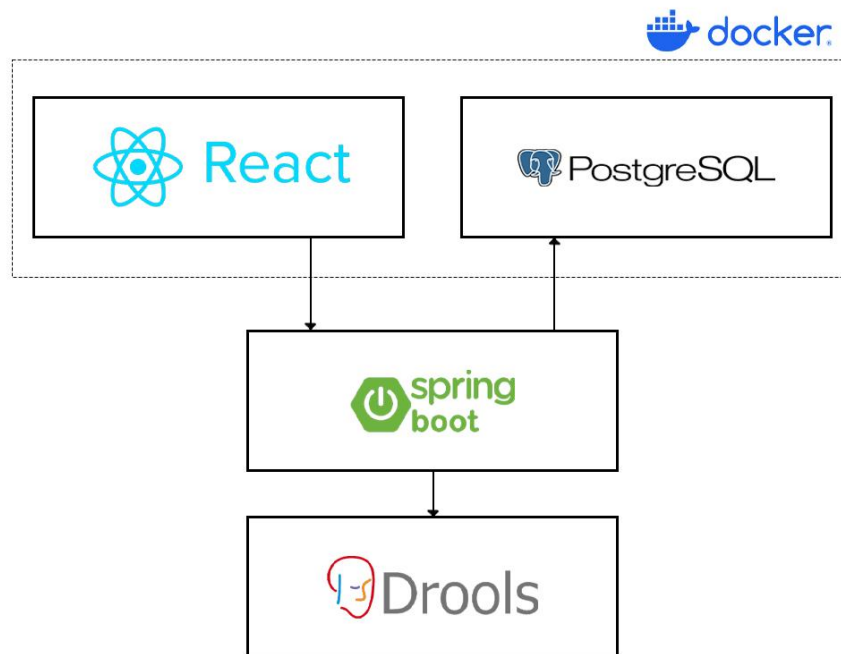
Иако су приручници били од велике помоћи у своје време, ограничење њиховог статичког приступа је постало очигледно са развојем алата који омогућују динамичке прорачуне и симулације у реалном времену.

2.8. Пресек стања

Тренутно стање показује јасну поделу између традиционалних, физичких водича и модерних алата. Водичи имају статички приступ и не омогућавају унос динамичких параметара, што ограничава њихову примену у конкретним ситуацијама. Са друге стране, савремени алати омогућавају симулације и калкулације, али се углавном фокусирају на индивидуалне борбе између две јединице, или појединачне аспекте игре попут енциклопедије јединица или калкулатора штете. Иако ове алатке помажу играчима у планирању и процени, тренутно не постоји решење које омогућава анализу целе битке са предлогом оптималних потеза у реалном времену. Тај недостатак оставља простор за развој јединственог алата који би објединио динамичке прорачуне, симулацију целих битака и давање препорука у току игре.

3. Коришћене технике и технологије

Пројекат је имплементиран као веб апликација, која се састоји од клијентске стране, серверске стране и базе података. На клијентској страни веб апликације користи се *React* [13], док је серверска страна развијена у оквиру *Spring Boot-a* [14]. Серверска апликација комуницира са базом података имплементираном у *PostgreSQL* [15]. За имплементацију и управљање пословном логиком користи се *Drools* [16]. Клијентска апликација и база података су интегрисани у *Docker* контејнере [17] ради једноставније конфигурације и покретање система. На слици [Слика 3.1](#) приказана је архитектура система.



Слика 3.1 - Архитектура система

3.1. React

React је *JavaScript* библиотека креирана од стране *Facebook-a* која служи за израду корисничког интерфејса. *React* омогућава да се *HTML* код смешта унутар *JavaScript-a* и користи виртуелним *DOM*, где се све потребне измене примењују у меморији пре прослеђивања на *DOM* претраживача. Уместо ажурирања целог *DOM-a*, *React* врши измене само над оним елементима који су заиста промењени [18].

Омогућава писање независних компоненти у виду *JavaScript* или *TypeScript* класа, које се касније лако комбинују за креирање страница. На овај начин се постиже скалабилност, поновна употребљивост и једноставно одржавање кода. Свака *React* компонента има свој животни циклус, који је дефинисан као скуп метода које се аутоматски извршавају у одређеним моментима током постојања компоненте [19].

React омогућава управљање подацима везаним за компоненту путем *state* објекта [20]. Подаци смештени у *state* доступни су искључиво унутар компоненте којој припадају, док се прослеђивање тих података дечјим компонентама врши преко *props* параметра. За разлику од *state-a* који је мутабилан и чије вредности је могуће мењати у оквиру компоненте, *props* је сам по себи имутабилан, и дечија компонента која га прима не може мењати његову вредност. Уз помоћ *state* и *props* омогућавамо јасну контролу тока података и доприносе поновној употребљивости и динамичком генерисању садржаја компоненти.

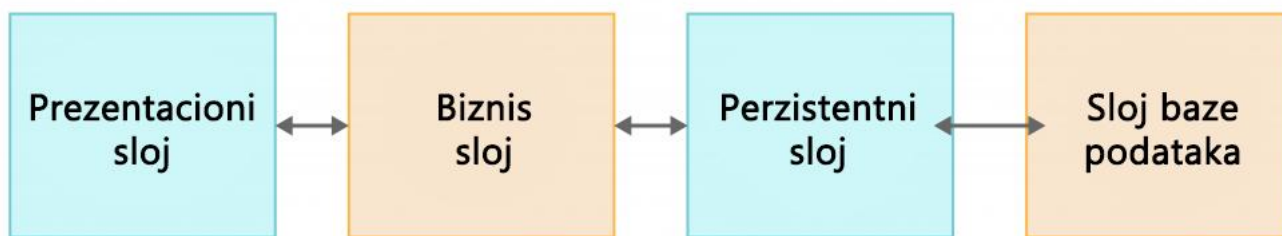
Верзија *React*-а која је коришћена за развој корисничког интерфејса је 18.3.1.

3.2. Spring Boot

Spring Boot је *Java* оквир који служи за лакше креирање и покретање *Java* апликација. Поједностављује конфигурацију и подешавање пројекта што омогућава програмерима да се фокусирају више на писање кода уместо подешавања окружења. Представља надоградњу у односу на оригиналан *Spring* оквир, где *Spring* омогућава флексибилност приликом израде апликација, док *Spring Boot* поједностављује све то и убрзава сам развојни процес. *Spring Boot* обезбеђује ауто-конфигурацију која аутоматски подешава све потребне компоненте на основу додатих зависности, користећи уграђене сервере попут *Tomcat*-а и *Jetty*-ја.

У сржи *Spring Boot*-а се налазе концепти инверзије контроле (*Inversion of Control*) и убризгавања зависности (*Dependency Injection*), који мењају начин одржавања зависности и креирање објеката у *Java* апликацији [21]. Инверзија контроле је дизајн принцип у којем се одговорност за креирање и управљање објектима не налази у самим класама, већ се тај задатак препушта *IoC* контејнеру. Уместо да класе саме иницирају и увезују своје зависности, контејнер то ради уместо њих. Убризгавање зависности је практична примена овог принципа, где када једна класа зависи од друге, контејнер аутоматски увезује ту зависност уместо програмера. Имплементацијом *IoC*-а и *DI*-а олакшавамо поновну употребљивост одређених компоненти унутар апликације, као и њихово лакше одржавање и тестирање. Поједине компоненте се могу лакше мењати, надограђивати или тестирати изоловано, без утицаја на остатак апликације.

Архитектура *Spring Boot* апликације најчешће се састоји из четири слоја: презентациони слој, бизнис слој, перзистентни слој и слој базе података [22]. Презентациони слој је одговоран за комуникацију са клијентом, и садржи контролере који обрађују *HTTP* захтеве и враћају одговоре. Бизнис слој представља централни део апликације у којем се налази пословна логика, где се дефинишу правила пословања и операције које апликација извршава. Перзистентни слој је задужен за управљање приступом подацима, и у њему се налазе репозиторијуми и механизми за рад са базом података попут *Hibernate* и *JPA*. Слој базе података чини саму базу података у којој се трајно чувају сви подаци апликације. Слојеви *Spring Boot* архитектуре се налазе на слици [Слика 3.2](#).



Слика 3.2 - Архитектура *Spring Boot* апликације [22]

Апликација је креирана као *Spring Boot Maven* пројекат, што значи да се за управљање зависностима, изградњу и паковање користи *Maven*, један од најзаступљенијих алата у *Java* екосистему [23]. *Maven* омогућава стандардизован начин организације пројекта, аутоматско преузимање и управљање библиотекама, као и дефинисање процеса изградње. На тај начин развојни тим добија јасну структуру, једноставнију контролу над верзијама зависности и могућност лаког проширивања пројекта новим модулима.

Сваки модул унутар система има свој засебан *pom.xml* фајл, у којем се налазе све потребне зависности, као и конфигурације специфичне за тај део апликације. На нивоу целог пројекта, *Maven* омогућава да се ови модули интегришу у једну целину, тако да развој и одржавање постају прегледнији и мање склони грешкама.

Поред тога, овакав приступ олакшава и континуирану интеграцију и *deployment*, јер свака промена у коду може аутоматски да се тестира, изгради и имплементира у радно окружење.

За креирање *Spring Boot* пројекта коришћен је *Spring Initializr* [24], а верзија *Jave* је 22.

3.3. Drools

Drools је систем за управљање пословним правилима (*BRMS*) који се користи за аутоматизацију пословних правила у софтверу. Правила се пишу у декларативном облику на *Drools Rule Language (DRL)*, и чувају се у засебним фајловима, што нам пружа лакше одржавање пословне логике без рекомпајлирања апликације. Такође пружа *rule engine* који ефикасно обрађује велику количину правила и враћа резултат у реалном времену.

Међу најбитнијим основним појмовима *Drools* система се налази чињеница (*Fact*), што представља податак који се користи за покретање правила. Правило (*Rule*) повезује одређене чињенице са одређеним акцијама. База знања (*Knowledge Base*) представља централизовани репозиторијум у којем се чувају сва дефинисана правила и модели података. Кроз *Knowledge Session* се ради са базом, где сесија омогућава учитавање података и њихово повезивање са тренутно битним правилима. Прво се сва правила убаце у сесију, потом се правила која се поклапају покрећу. Модул (*Module*) садржи у себи више база знања која у себи могу чувати различите сесије [25].

Сваки *DRL* фајл садржи скуп правила која се пишу у декларативном облику и организују у пакете, чиме се обезбеђује логика повезаности између правила. Пример *DRL* фајла се може видети на слици [Слика 3.3](#). Свако правило у себи садржи свој идентификатор (*namespace*) који дефинише блок тог правила. Свако правило садржи *LHS (Left Hand Side)* и *RHS (Right Hand Side)*. *LHS* је део правила где се дефинишу услови који морају бити испуњени да би се правило активирало. *RHS* је део правила где се дефинише акција која ће бити извршена ако су сви услови из *LHS* испуњени. Уколико више правила испуњава услов са *LHS*, приоритет се одређује према раније дефинисаном приоритету, према позицији правила, дефинисаним мета правилима или према томе које правило је најскорије додато [26].

```
rule "name"
  attributes
  when
    LHS
  then
    RHS
end
```

Слика 3.3 - Пример *DRL* фајла [26]

Drools омогућава извршавање правила кроз различите начине, где су најзначајнији *forward chaining* и *backward chaining* [27]. *Forward chaining* почиње са познатим чињеницама и активно примењује правила за генерисање нових. Механизам процењује све доступне податке, активира правила која су повезана и наставља овај процес све док не остане без правила да покреће. *Backward chaining* има обрнут приступ, где уместо да се започне са подацима, започиње се са циљем. *Engine* потом ради уназад да пронађе која правила и чињенице подржавају тај циљ.

4. Спецификација система

Главни циљ овог система је да препоручи новим корисницима најоптималнији потез унутар игре *Heroes of Might and Magic III*, узимајући у обзир све релевантне параметре попут позиције јединица, доступне магије и редослед потеза. За разлику од постојећих динамичких система који углавном нуде анализу појединачних битки између две јединице, тренутни систем обухвата целокупни контекст битке, што му омогућава да предложи оптималне потезе узимајући у обзир све доступне информације на бојишту. Систем омогућава кориснику да донесе стратешки најисплативије одлуке у реалном времену, док постепено стиче дубље разумевање комплексних механика игре.

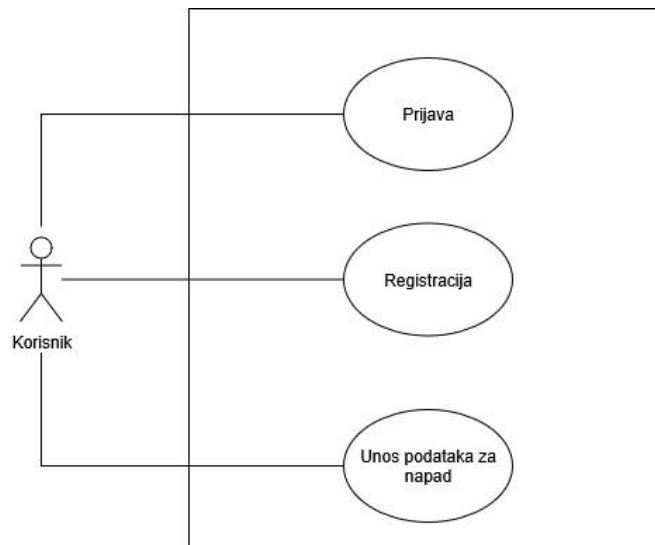
Спецификација ће дати преглед читаве апликације са високог нивоа, користећи UML (*Unified Modeling Language*) дијаграме. Дијаграми у питању су: дијаграм случајева коришћења, дијаграм класа и дијаграм секвенци.

4.1. Дијаграм случајева коришћења

Дијаграм случајева коришћења представља један од основних UML дијаграма који приказује функционалности система из перспективе корисника [28]. Примарна улога овог дијаграма јесте приказ могућности и функционалности доступне корисницима у интеракцији са системом, без детаља о самој имплементацији. Дијаграм не дефинише како се функције реализују, већ само шта је доступно актерима, чиме обезбеђује висок ниво апстракције и фокусира се на корисничку перспективу.

На дијаграму случајева коришћења, на слици [Слика 4.1](#), приказане су акције које су доступне кориснику, а то су:

- Пријава - Корисник уноси свој мејл и лозинку ради пријаве.
- Регистрација - Корисник уноси свој мејл, корисничко име и лозинку ради креирања налога.
- Унос података за напад - Корисник уноси све податке из битке и прослеђује их систему да одлучи који је најбољи потез за одиграти. Унос је могућ тек након пријаве у систем.



Слика 4.1 - Дијаграм случајева коришћења апликације

4.2. Дијаграм класа

Дијаграм класа је тип дијаграма у *UML*-у који описује структуру система, приказујући класе, њихове атрибуте, методе и релације међу њима [29]. Дијаграм класа представља основни елемент објектно оријентисаног моделовања, јер омогућава разумевање статичке архитектуре система. Пружа јасан преглед компоненти и њихових међусобних односа, и често представља основу за генерисање кода или документовање већ постојећих система.

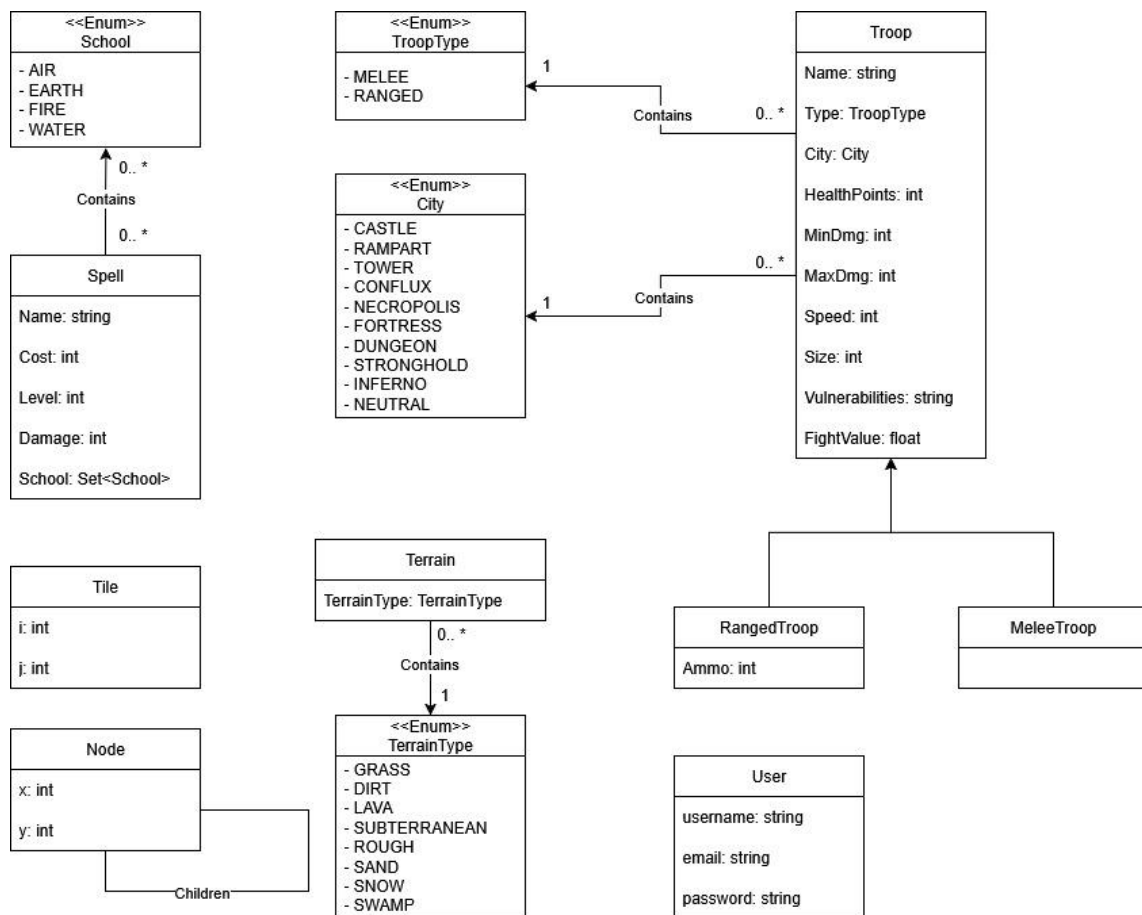
Ради лакшег прегледа, набрајају се само класе које нису креиране специфично за *Drools* правила као и *DTO* класе, већ ћемо се фокусирати на основне modele и енумерације које чине структуру апликације. Дијаграм класа апликације приказан је на слици [Слика 4.2](#). Модел апликације чини седам модела и четири енумерације.

Идентификоване класе су:

- *Troop* – апстракција трупа у систему. Свака трупа описана је именом (поље *name*), типом (поље *type*), градом (поље *city*), животним поенима (поље *healthpoints*), минималном штетом и максималном штетом коју трупа може нанети (поља *mindmg* и *maxdmg*), брзином (поље *speed*), величином (поље *size*), рањивостима (поље *vulnerabilities*) као и борбеном јачином (поље *fightvalue*). Ову класу наслеђују две подкласе: *MeleeTroop*, као и *RangedTroop* са параметром за муницију (поље *ammo*).
- *Spell* – апстракција магија у систему. Свака магија описана је именом (поље *name*), ценом (поље *price*), нивоом (поље *level*), штетом (поље *damage*) и школом (поље *school*).
- *Terrain* – апстракција терена у систему. Описан је енумерацијом типа терена (поље *terraintype*)
- *Tile* – апстракција поља на ком се трупа налази.
- *Node* – апстракција поља на терену. Свако поље је описано координатом *x* и *y*, као и листом поља околних око оригиналног поља (поље *children*).
- *User* – подаци корисника. Сваки корисник је описан корисничким именом (поље *username*), мејлом (поље *email*) и шифром (поље *password*).

Идентификоване енумерације су:

- *School* – тип школе којој магија припада. Могуће вредности су *AIR*, *EARTH*, *FIRE* и *WATER*.
- *City* – град којем одређена трупа припада. Могуће вредности су *CASTLE*, *RAMPART*, *TOWER*, *CONFLUX*, *NECROPOLIS*, *FORTRESS*, *DUNGEON*, *STRONGHOLD*, *INFERNO* и *NEUTRAL*.
- *TerrainType* – тип терена на ком се битка одиграва. Могуће вредности су *GRASS*, *DIRT*, *LAVA*, *SUBTERRANEAN*, *ROUGH*, *SAND*, *SNOW* и *SWAMP*
- *TroopType* – тип трупе на терену. Могуће вредности су *MELEE* и *RANGED*.

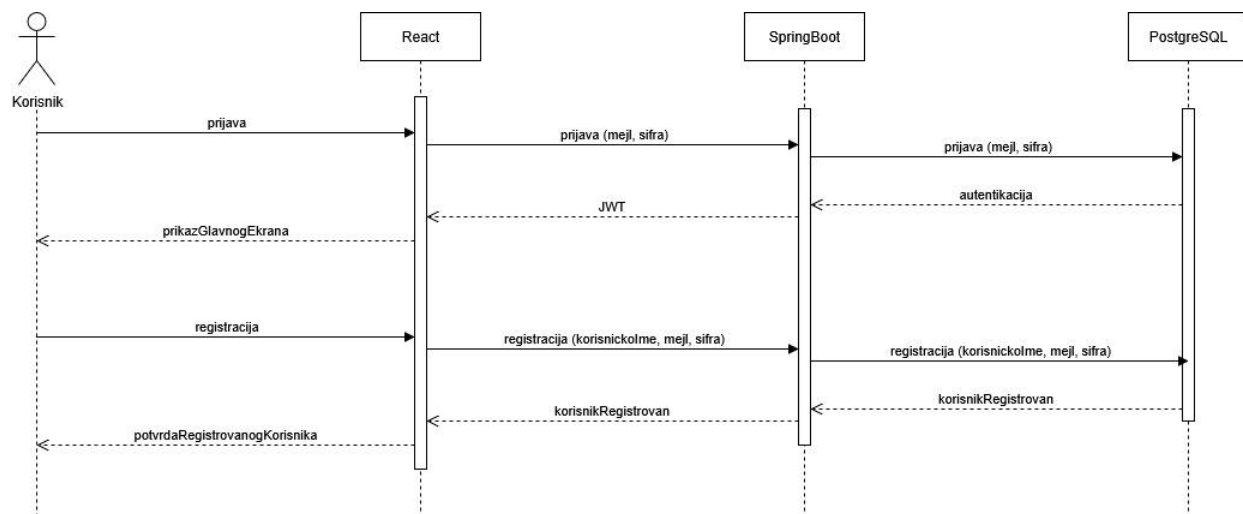


Слика 4.2 - Дијаграм класа апликације

4.3. Дијаграм секвенце

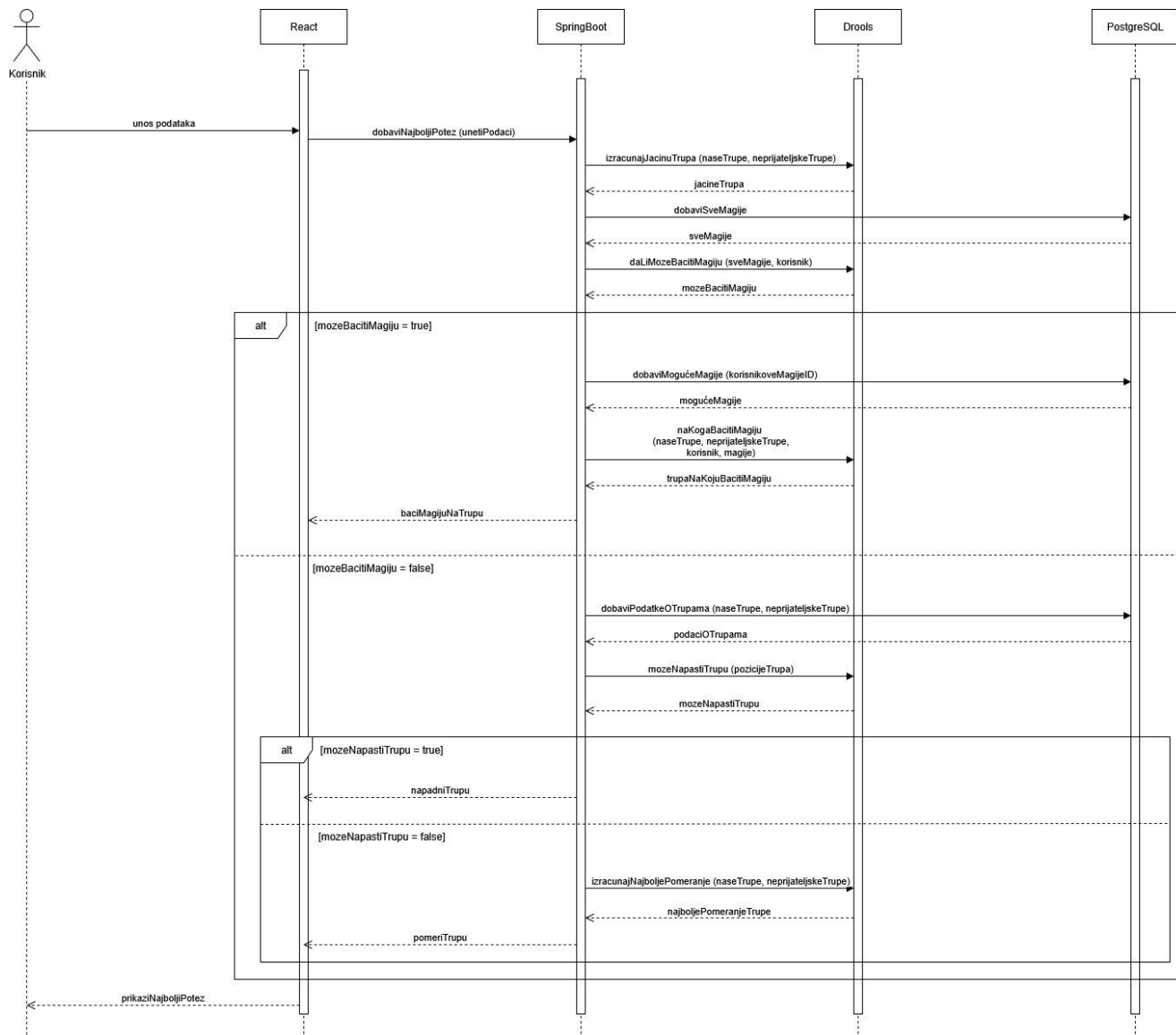
Дијаграм секвенце је *UML* дијаграм који приказује како објекти у систему међусобно комуницирају у одређеном временском редоследу [30]. Његова главна сврха јесте да прикаже понашање система кроз размену порука између објеката, уз јасан редослед прослеђивања порука. У овом дијаграму објекти су представљени као вертикалне линије (линије живота), док су поруке између њих приказане хоризонталним стрелицама које указују на ток комуникације. Редослед догађаја се прати од врха ка дну. Дијаграми секвенци се најчешће користе у анализи и дизајну система, јер приказују како се функционалности из дијаграма случајева коришћења спроводе кроз интеракцију објеката. Осим у дизајнирању нових система, ови дијаграми се често користе и за документовање постојећих система, јер омогућавају лакше разумевање сложених процеса.

На дијаграму секвенци су приказане функционалности пријаве, регистрације као и доношење најоптималнијег потеза у оквиру борбе. Корисник прво приступа *React* апликацији где може да изврши пријаву или регистрацију. Уколико је пријава успешна, систем враћа *JWT* [31], чиме се омогућава сигурна комуникација са *Spring Boot*-ом. Слика [Слика 4.3](#) приказује дијаграм секвенци за процес пријаве и регистрације.



Слика 4.3 - Дијаграм секвенце пријаве и регистрације корисника

Након тога, корисник уноси податке потребне за израчунавање оптималног потеза, који се прослеђују *Spring Boot* апликацији. *Spring Boot* потом покреће логику за израчунавање најбољег потеза, укључујући добављање свих магија, одабир могућих магија и проверу услова кроз *Drools rules engine*. Ако постоји могућност употреба магије, систем калкулише на коју јединицу треба бацити магију и шаље одговарајући одговор *React* клијенту. Уколико магија није доступна, извршава се прорачун за напад или померање јединица, где се прво проверава да ли је могућ директан напад на непријатељску трупу. Уколико није, систем израчунава најоптималније померање трупе. На крају, *React* клијент добија информацију о најбољем потезу, као и процена чија трупа је јача, што се све приказује кориснику. Слика [Слика 4.4](#) приказује дијаграм секвенци за овај процес.



Слика 4.4 - Дијаграм секвенце израчунавања оптималног потеза

5. Имплементација система

У овом поглављу биће представљена имплементација система заснованог на архитектури описаној у претходним поглављима. Систем је реализован као комбинација три целине – клијентске апликације развијене у *React*-у, серверске апликације развијене у *Spring Boot*-у и система заснованог на правилима имплементиран уз помоћ *Drools*-а. У наставку ће бити описан начин реализације сваког дела система, као и начин на који су појединачни делови система повезани у целину.

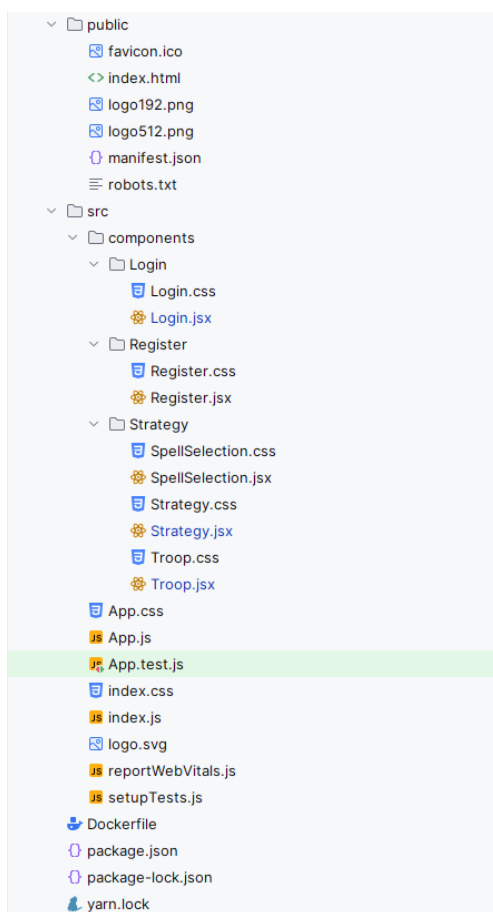
Комплетан изворни код система доступан је на *GitHub* платформи, на следећем линку: https://github.com/slepimis120/Stefan_Nemanja.

5.1. Клијентска апликација

Клијентска апликација је имплементирана користећи *React* радни оквир. Компоненте апликације су логички раздвојене и груписане по начину примене. Свакој компоненти је додељена јасна одговорност, чиме се омогућава лакше одржавање и проширење апликације у будућности. Навигација у апликацији је једноставна и интуитивна, омогућавајући корисницима лак приступ свим доступним функцијама.

Целине апликације се може видети на слици [Слика 5.1](#). и чине је следеће компоненте:

- *Login*
- *Register*
- *Strategy*



Слика 5.1 - Структура клијентске апликације

Login компонента за улогу има пријаву корисника у сам систем. Састоји се од поља за унос мејла, шифре, и дугме за саму пријаву. Кликком на дугме се шаље захтев *Spring Boot*-у преко *fetch* методе, преко које добијамо *JWT*. *JWT* се складишти у *SessionStorage*-у, и шаље се у *header*-у нових захтева ради аутентификације и ауторизације. Уколико корисник нема претходно креиран налог, такође постоји опција да се са *Login* странице корисник преусмери на *Register* страницу.

Register компонента за улогу има регистровање нових корисника. Од података који се траже за регистрацију, узима се корисничко име, мејл и шифра. Након регистрације се корисник враћа на *Login* страницу како би се улоговао са новокреираним налогом.

Strategy је главна компонента апликације, чија је сврха да кориснику омогући унос детаља битке како би систем могао да предложи оптималан потез. Корисник унутар ове компоненте има могућност дефинисања више параметара битке, укључујући податке о пријатељским и непријатељским јединицама, избор доступних магија, дефинисање типа терена на ком се битка одвија, као и одређивање броја поена за коришћење магија. Сама страница је подељена на три логичке целине:

- *Strategy*
- *Troop*
- *SpellSelection*

У оквиру те структуре, *Strategy* компонента служи као родитељска компонента која у себе увезује две дечје компоненте: *SpellSelection* и *Troop*, интегришући их у сам интерфејс који се приказује кориснику. Сама компонента управља сетом локалних стања која обухватају податке о јединицама и магијама који су потребни за слање захтева ка серверу. Локална стања се континуирано ажурирају и синхронизују са дечјим компонентама, чиме родитељска компонента увек поседује нове и потпуне информације о свим елементима битке. Приликом учитавања саме компоненте, коришћењем *useEffect hook*-а, апликација аутоматски преузима све податке о јединицама и магијама са сервера. Сваки захтев садржи *JWT* унутар *header*-а, што омогућава да само валидни и аутентификовани корисници имају приступ потребним информацијама. Компонента такође омогућава динамичко додавање и уклањање јединица са листе, као и управљање редоследом и бројем јединица у свакој категорији. Када корисник заврши са уносом свих потребних информација и жели да добије препоруку за следећи потез, сви унесени подаци се шаљу серверу *POST* методом, након чега сервер на основу примљених информација израчунава најоптималнији потез.

Troop компонента за циљ има да омогући кориснику унос и праћење података о појединачним јединицама унутар саме битке. Компонента је дизајнирана као контролисана *React* компонента која комуницира са родитељском *Strategy* компонентом. Основна сврха компоненте је да омогући кориснику да унесе све релевантне параметре јединице потребне за израчунавање оптималног потеза у оквиру битке. Компонента прима следеће *props*-ове од *Strategy* компоненте: листу доступних типова јединица, функцију за уклањање јединице, редни број јединице у низу, класу која одређује припадност јединице (да ли је јединица пријатељска или непријатељска), јединствени идентификатор јединице, и *callback* функцију која шаље ажуриране податке родитељској компоненти. Локална стања компоненте укључују тип и назив јединице, број јединица, здравље, позицију на мапи (уз помоћ координата *i* и *j*), количину муниције за *RANGED* типове јединице, као и променљиве које означавају да ли је јединица у покрету или је већ чекала свој потез. Свака промена ових стања аутоматски се прослеђује родитељској компоненти кроз *callback*, што омогућава да *Strategy* компонента увек има ажуране податке о свим јединицама.

SpellSelection компонента за циљ има да омогући кориснику избор доступних магија које се могу користити у оквиру битке. Основна сврха компоненте је да прикаже све доступне магије и омогући кориснику да селекује једну или више магија које жели да користи. Компонента прима три *props*-а од *Strategy*

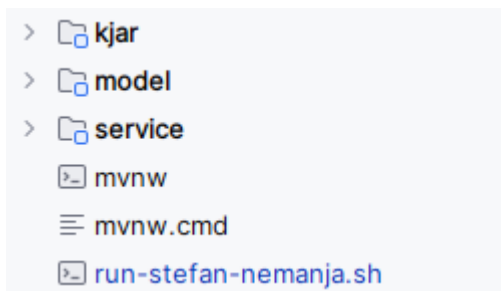
компоненте: листу свих доступних магија (*spells*), тренутно изабране магије (*selectedSpells*) и *callback* функцију (*setSelectedSpells*) која омогућава ажурирање избора у родитељској компоненти. Корисник може кликом на поједину магију означити њен избор или уклонити већ изабрану магију са листе. Свака промена у избору магија аутоматски се прослеђује родитељској компоненти кроз *callback* функцију, чиме *Strategy* компонента увек има актуелне информације о томе које магије су доступне за употребу.

5.2. Серверска апликација

Серверска апликација је имплементирана користећи *Spring Boot* радни оквир, који омогућава брз развој модуларних и скалабилних система намењених производном окружењу. Пружа једноставну конфигурацију, интеграцију са другим библиотекама и алатима, као и висок степен флексибилности у прилагођавању специфичним потребама пројекта. Архитектура је осмишљена тако да свака целина има јасно дефинисану улогу, чиме се обезбеђује ефикасна обрада података, лакше тестирање и једноставније одржавање у дужем временском периоду.

У оквиру апликације издвојене су три главне целине, које се могу видети на слици [Слика 5.2](#) , и то су:

- *Model*
- *Kjar*
- *Service*



Слика 5.2 - Структура серверске апликације

Иако су целине креиране као засебни *Spring Boot* пројекти, ови модули међусобно комуницирају и узајамно се повезују унутар пројектног окружења [32]. На тај начин свака апликација задржава сопствену аутономију у развоју, конфигурацији и одржавању, али да притом свака својим функцијама доприноси заједничком решењу. Раздвајање на више независних *Spring Boot* пројеката омогућава јасно дефинисану модуларну архитектуру. Уколико је потребно изменити или проширити функционалности једног модула, то се може извршити без директног утицаја на друге делове система, чиме се смањује ризик од грешака и убрзава процес развоја.

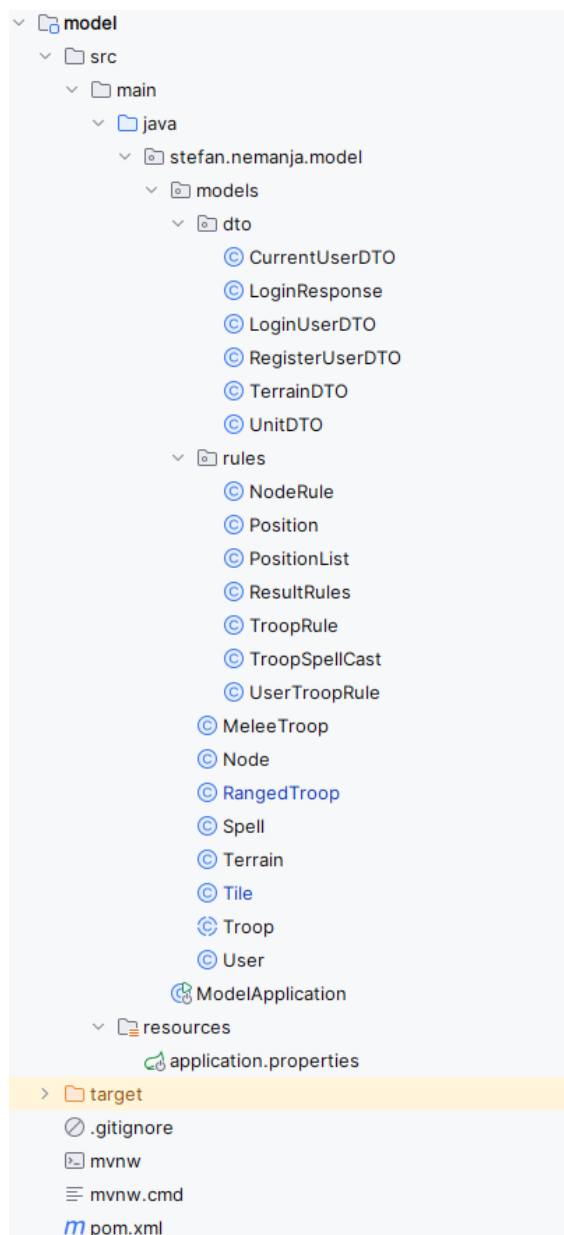
Још једна значајна предност јесте могућност скалабилности. Сваки пројекат се може независно развијати, *deploy*-овати и по потреби надограђивати новим верзијама, без застоја у раду других делова система. Оваква архитектура обезбеђује већу отпорност на потенцијалне проблеме, јер евентуални квар једног модула не мора да доведе до застоја целог система.

5.2.1. Model компонента

Model пројекат представља темељ целокупног система. У њему се дефинишу сви ентитети и њихови односи, што чини основу доменске логике апликације. *Model* обезбеђује јасну и стабилну структуру података и смањује могућност грешака током даљег развоја. Представља ослонац система, централно место које гарантује да све остале компоненте међусобно комуницирају

С обзиром да се ради о заједничком слоју, *Model* користе и остали пројекти у систему – *Kjar* и *Service*. Овим приступом сви модули раде над истим апстракцијама и моделима, што пружа уједначеност у раду, једноставнију интеграцију између појединих делова и већу транспарентност унутар целог пројекта. Такође, уколико се јави потреба да се структура прошири новим ентитетима или да се измене постојеће везе, довољно је да се измене изврше само у *Model* пројекту, а све друге компоненте ће имати приступ новој, ажурираној верзији.

Структура пројекта се може видети на слици [Слика 5.3](#).



Слика 5.3 - Структура *Model* компоненте

Сваки од пројеката у систему је *Maven* апликација, што значи да је њихова структура дефинисана кроз *pom.xml* фајл. Овај фајл представља централно место за конфигурацију, јер у њему стоје информације о зависностима, верзијама библиотека, *plugin*-има који се користе током изградње и подешавањима

специфичним за дати пројекат. Тренутно су у *Model*-овом *pom.xml*-у наведене библиотеке неопходне за рад са *Spring Boot*-ом, *JPA* репозиторијумима, *PostgreSQL* базом и *Drools engine*-ом.

Унутар овог *pom.xml* фајла дефинисане су све потребне зависности за рад *Model* пројекта, и то су:

- *spring-boot-starter-data-jpa* – омогућава једноставну имплементацију репозиторијума и рад са базом података преко *JPA* стандарда, што је кључно за креирање и управљање ентитетима у *Model* слоју. Верзија коришћена у пројекту је 4.0.0-M2.
- *spring-security-core* – обезбеђује безбедносне механизме и класе које се могу користити за контролу приступа и управљање корисничким подацима. Верзија коришћена у пројекту је 7.0.0-M2.
- *kie-api* – централна библиотека *Drools* система која дефинише интерфејсе и *API* за рад са пословним правилима и *rule engine*-ом. Верзија коришћена у пројекту је 10.1.0.

У *Model* пројекту, рад са ентитетима је организован кроз коришћење *JPA* анотација, што омогућава директно мапирање *Java* класа на релационе базе података [33]. Основна идеја је да свака класа која представља ентитет буде обележена анотацијом *@Entity*, чиме се говори да ће за њу постојати одговарајућа табела у бази. Опционална анотација *@Table* се користи за дефинисање имена табеле. На пример, класа *User* се мапира на табелу *users*. Поља попут *id*, *email* и *username* користе анотацију *@Id*, *@GeneratedValue* и *@Column* за дефинисање примарног кључа, аутоматског генерисања вредности и ограничења на колоне.

За рад са *enum* типовима и колекцијама користимо *@Enumerated* и *@ElementCollection*. На пример, уз класи *Troop* поље *type* и *city* је типа *enum* и користи *@Enumerated(EnumType.STRING)*, док класа *Spell* има поље *School* које је скуп *enum* вредности и реализовано је преко *@ElementCollection* и *@CollectionTable*. Ово омогућава лакше чување и читање сложених структура података без додатних помоћних табела или ручног мапирања.

У *Model* пројекту, класа *User* представља ентитет који комбинује *JPA* и *Spring Security* функционалности [34]. Она је означена анотацијом *@Entity* и мапирана на табелу *users* преко *@Table(name = "users")*, што омогућава да сваки корисник буде сачуван у релационој бази података са јасно дефинисаним пољима попут *id*, *username*, *email* и *password*. Анотације *@Id* и *@GeneratedValue* аутоматски генеришу идентификатор за сваког корисника, док *@Column* дефинише ограничења и јединственост појединих поља. Класа *User* се може видети на слици [Слика 5.4](#).

```

@Table(name = "users")  @slepimis120
@Entity
public class User implements UserDetails{
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    @Column(nullable = false)
    private Integer id;

    @Column(nullable = false)  2 usages
    private String username;

    @Column(unique = true, length = 100, nullable = false)  4 usages
    private String email;

    @Column(nullable = false)  3 usages
    private String password;

    public User() {  @slepimis120
    }

    public User(String username, String email, String password) {  1 usage  @slepimis120
        this.username = username;
        this.email = email;
        this.password = password;
    }
}

```

Слика 5.4 - User класа

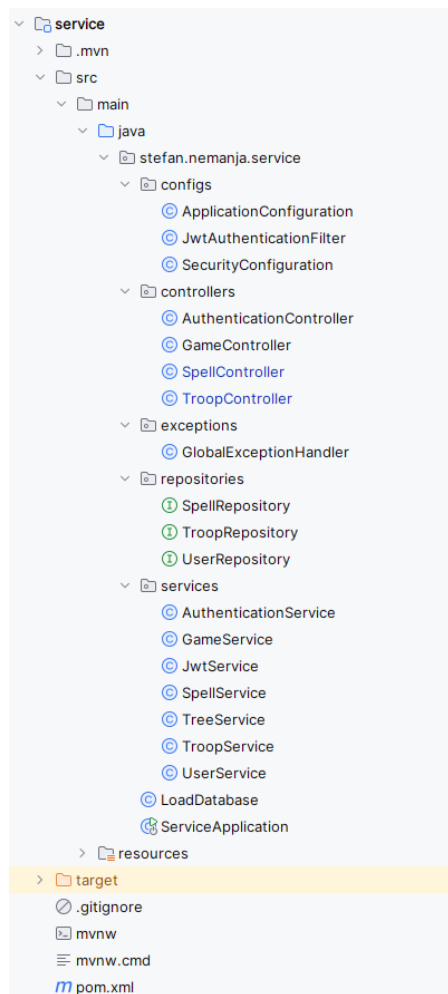
Имплементација интерфејса *UserDetails* омогућава да *User* ентитет директно учествује у механизму аутентификације и ауторизације у оквиру *Spring Security*-а. Метода *getUsername()* враћа мејл као идентификатор за пријаву, а методе *isAccountNonExpired()*, *isAccountNonLocked()*, *isCredentialsNonExpired()* и *isEnabled()* дефинишу статус корисничког налога. У самој имплементацији, све ове методе враћају *true*, што значи да сви налози подразумевано имају активан и неограничен приступ систему.

5.2.2. Service компонента

Service пројекат представља слој који енкапсулира целокупну пословну логику апликације. За разлику од *Model* пројекта, који дефинише структуру података и ентитете, *Service* се бави обрадом тих података, координацијом између различитих компоненти и обезбеђивањем функционалности апликације. Он користи ентитете из *Model* пројекта и пружа методе за креирање, ажурирање и управљање њиховим стањем, као и за имплементацију правила и услова специфичних за домен апликације.

Структура *Service* пројекта се може видети на слици [Слика 5.5](#), и организована је по пакетима који раздвајају одговорности:

- *Configs* – садржи конфигурације апликације и безбедносне филтере који управљају аутентификацијом и приступом ресурсима.
- *Controllers* – омогућава приступ функционалностима апликације преко *REST* интерфејса и комуникацију са клијентским захтевима.
- *Services* – имплементира стварну пословну логику, повезујући ентитете и репозиторијуме у смислене операције.
- *Repositories* – омогућава комуникацију са базом података и управља перзистенцијом ентитета
- *Exceptions* – обрађује грешке и изузетке, осигуравајући стабилност и контролу током рада апликације



Слика 5.5 - Структура *Service* компоненте

Када корисник на клијентској апликацији иницира захтев за најбољи потез у игри, подаци се шаљу преко *REST* позива ка ендпоинту */strategy/bestMove*. Контролер *GameController* прима захтев у виду објекта *CurrentUserDTO*, који садржи информације о корисниковим трупима, расположивим магијама и тренутном стању игре. Контролер не обрађује саму пословну логику, већ прослеђује податке сервисном слоју и враћа резултат у виду објекта *ResultRules*.

Сервисни слој, који представља срж пословне логике апликације, прво проверава доступне магије и креира *Drools* сесију на бази правила *forwardKbase*. Подаци о кориснику и све релевантне магије убацују се у сесију, након чега се правила извршавају. На основу стања корисника и резултата правила, сервис одлучује да ли ће се бацити магија или ће се трупе кретати према најбољем потезу. Ако је магија доступна, метод *castSpell* из *SpellService* обрађује ефекте магијског напада. Уколико магија није могућа, *TroopService* одређује оптималан покрет трупа, користећи *Drools* сесију *bwBase* за процену могућности напада и рангирање непријатеља на основу позиције, здравља и броја јединица. Овај процес је илустрован на слици [Слика 5.6](#), где је приказан део кода са методом *getBestMove* који интегрише *Drools* правила и логичку обраду, показујући ток од уноса података до доношења одлуке о следећем потезу.

```

public ResultRules getBestMove(CurrentUserDTO currentUserDTO) { 1 usage  👤 slepimis120
    List<Spell> spells = spellService.getSpellsByIds(currentUserDTO.getAvailableSpells());

    KieSession kieSession = kieContainer.getKieBase( s: "forwardKbase").newKieSession();
    kieSession.getAgenda().getAgendaGroup( s: "spell-check").setFocus();
    kieSession.insert(currentUserDTO);

    for (Spell spell : spells) {
        kieSession.insert(spell);
    }

    kieSession.fireAllRules();
    kieSession.dispose();

    ResultRules rule;

    if (currentUserDTO.isCanCastSpell()) {
        rule = spellService.castSpell(currentUserDTO);
    }
    else{
        rule = troopService.bestMove(currentUserDTO);
    }

    ResultRules troopStrength = getTroopStrength(currentUserDTO);
    rule.setResult(rule.getResult() + "\n" +troopStrength.getResult());
    return rule;
}

```

Слика 5.6 - Логика за одабир најбољег потеза

Репозиторији *SpellRepository* и *TroopRepository* обезбеђују слој перзистенције и омогућавају сервисима приступ подацима из базе, без директног ангажовања базе у самом сервисном слоју. На овај начин, *Service* пројекат интегрисхе податке, пословна правила и логичку обраду у једну компактну функционалност, омогућавајући кориснику да добије препоруку за најбољи потез у реалном времену.

Унутар *pom.xml* фајла дефинисане су све основне зависности потребне за рад *Service* пројекта, а најбитније од њих су:

- *spring-boot-starter*, *spring-boot-starter-web*, *spring boot-starter-security* – обезбеђују основну инфраструктуру *Spring Boot* апликације, подршку за креирање веб сервиса и безбедносне механизме за контролу приступа и аутентификацију корисника.
- *spring-security-jwt*, *io.jsonwebtoken (jjwt-api, jjwt-impl, jjwt-jackson)* – омогућавају рад са *JWT* токенима, што је кључно за имплементацију аутентификације и ауторизације у апликацији.
- *org.postgresql:postgresql* – *JDBC* драјвер за повезивање са *PostgreSQL* базом података.
- *org.kie:kie-api*, *org.kie:kie-ci*, као и *drools-core*, *drools-compiler*, *drools-tempaltes*, *drools-decisiontables*, *drools-xml-support* – централне библиотеке *Drools* система, које омогућавају креирање и извршавање пословних правила у *Service* слоју.
- *org.apache.poi:poi*, *poi-ooxml* – библиотека за рад са *Excel* документима и другим *Office* форматима.

5.2.3. Kjar компонента

Kjar пројекат представља посебан слој система намењен дефинисању и управљању пословним правилима. За разлику од *Service* пројекта, који садржи имплементацију логике и позивање метода, *Kjar*

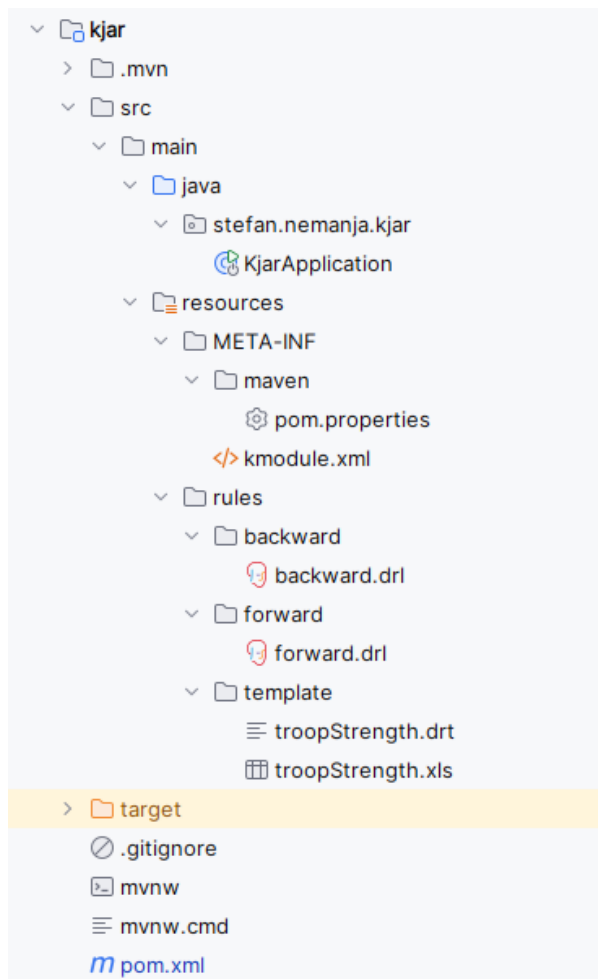
садржи формалне декларације правила у оквиру *Drools engine*-а. Овај модул је замишљен као независни артефакт у коме се налазе *DRL (Drools Rule Language)* фајлови, шаблони, одлуке и друге конфигурације које описују начин на који се логика игре примењује у различитим ситуацијама.

Основна предност овог приступа је раздвајање између статичког дела апликације (ентитети, сервиси, базе) и динамичког дела (пословна правила). Док се *Service* бави конкретном обрадом и интеграцијом података, *Kjar* омогућава да се правила лако мењају, додају или уклањају без потребе за компилацијом и изменом самог кода апликације. На тај начин се добија систем који је флексибилан, одржив и прилагодљив променама у пословној логици.

Kjar у овом систему садржи сва правила која управљају понашањем јединица, магија или стратегијом кретања. Она се извршавају унутар *Drools* сесија које *Service* креира и иницира, али сам процес доношења одлука остаје у потпуности дефинисан унутар овог модула. Овим приступом се постиже јасно раздвајање, у *Kjar*-у се налази шта апликација треба да уради кроз правила, док је у *Service*-у описано на који начин ће се та правила спровести у самој логици.

Структура пројекта се може видети на слици [Слика 5.7](#), и организована је на следећи начин:

- *META-INF* – садржи конфигурационе датотеке неопходне за покретање *Drools* механизма, попут *kmodule.xml*, у коме су дефинисане базе правила и сесије које *Service* слој користи.
- *rules* – представља централно место у коме су смештена сва пословна правила. Унутар овог директоријума налазе се под-директоријуми *forward*, *backward* и *template*, који групишу правила по начину извршавања или облику у коме су дефинисана
- *forward* – садржи правила која се извршавају напредним (*forward chaining*) механизмом закључивања, где свака нова чињеница може активирати додатна правила.
- *backward* – садржи правила заснована на обрнутом механизму (*backward chaining*), који почиње од циља и враћа се уназад како би утврдио да ли се услови могу испунити.
- *template* – обухвата шаблоне и табеле одлука (*.drt*, *.xls*), који омогућавају да се правила лако проширују и одржавају, без потребе за изменом самог кода.



Слика 5.7 - Структура *Kjar* компоненте

Forward chaining представља један од основних приступа у систему правила и користи се за аутоматско извођење закључака на основу познатих чињеница и дефинисаних правила. Систем креира радну меморију чињеница и пролази кроз њу, проверавајући сва правила и извршавајући она која се могу применити, при чему резултати могу да генеришу нове чињенице, што може покренути следећа правила. *Forward chaining* је посебно користан у ситуацијама када је потребно динамички реаговати на променљиве податке и доносити одлуке у реалном времену.

У контексту апликације, *forward chaining* се користи за одређивање могућности коришћења магије од стране хероја, за планирање кретања јединица и за одабир непријатељских трупа које треба напасти.

Пре него што се било која магија баци, систем проверава да ли корисник има довољно *SpellPoints* да изврши бар један од доступних магијских напада. На слици [Слика 5.8](#) приказано је правило “*Can user cast a spell*”, које проверава да ли тренутни корисник испуњава услов за бацање магије. Уколико корисник још није искористио своју магију, систем прикупља све доступне магије и одређује оне са најмањом ценом бацања. Затим се проверава да ли корисник има више *SpellPoints* од те минималне вредности. Уколико има, правило пролази и поставља се кориснику статус да може да баци магију.


```

rule "Can user cast a spell"
agenda-group "spell-check"
when
    $currentUser : CurrentUserDTO(usedSpell == false)
    $spells : List() from collect(Spell())
    $cheapestSpellPoints : Number() from accumulate (
        $spell : Spell() from $spells,
        min($spell.getCost())
    )
    eval($currentUser.getSpellPoints() > $cheapestSpellPoints.intValue())
then
    $currentUser.setCanCastSpell(true);
    update($currentUser);
end

```

Слика 5.8 - Правило за проверу могућности бацања магије

Након што се утврди да корисник може да баци магију, правила одређују редослед напада, при чему се приоритет даје непријатељским трупам које су најосетљивије на магију, затим *ranged* трупам, и на крају трупам са највећим здрављем. Уколико се испуне услови за ефикасан напад, систем проверава да ли постоје трупе које су осетљиве на магију, тако што се креира листа са непријатељским трупам и пореди се са магијама које корисник поседује [35]. На слици [Слика 5.9](#) приказано је правило “*Can cast spell on a vulnerable unit*”, које идентификује најјачу непријатељску трупу из групе оних које су подложне одређеној магији. Уколико таква трупа постоји, систем препоручује конкретну магију коју треба употребити на тој јединици, чиме се постиже максималан ефекат напада.

```

rule "Can cast spell on a vulnerable unit"
agenda-group "spell-casting"
salience 1
when
    $result : ResultRules()
    $enemyUserTroopRule : UserTroopRule(getIsFriendly() == false)
    $spells : List() from collect(Spell())
    $troop : TroopSpellCast() from $enemyUserTroopRule.getStrongestTroopWithVulnerabilityList($spells)
    eval($troop != null)
then
    $result.setResult("You should cast " + $troop.getSpellName() + " spell on " + $troop.getTroopName() + " because of spell vulnerability.");
    update($result);
end

```

Слика 5.9 - Бацање магије на рањиве јединице

Уколико не постоје трупе које су осетљиве на магију, систем прелази на следећи приоритет и проверава непријатељске трупе типа *ranged*, што се може видети на слици [Слика 5.10](#). Правило “*Can cast spell on a ranged unit*” идентификује најјачу *ranged* трупу на основу вредности максималне штете коју она може да нанесе. У првом кораку, систем одређује магију која има највећу јачину напада, а затим је упоређује са непријатељским *ranged* трупам како би изабрао циљ који представља највећу претњу. Ако таква трупа постоји, резултат правила је препорука да се баци магија са највећом јачином напада на одабрану *ranged* јединицу, чиме корисник добија највећу стратешку предност.

```

rule "Can cast spell on a ranged unit"
agenda-group "spell-casting"
salience 2
when
    $result : ResultRules()
    $enemyUserTroopRule : UserTroopRule(getIsFriendly() == false)
    $spells : List() from collect(Spell())

    $maxSpellDamage : Number() from accumulate(
        Spell( $damage : damage ),
        max($damage)
    )
    $maxSpell : Spell( damage == $maxSpellDamage )

    $maxTroopDamage : Number() from accumulate(
        TroopRule( getType() == Troop.TroopType.RANGED, $damage : getMaxDmg() ) from $enemyUserTroopRule.getTroops(),
        max($damage)
    )
    $maxTroop : TroopRule( getType() == Troop.TroopType.RANGED, getMaxDmg() == $maxTroopDamage ) from $enemyUserTroopRule.getTroops()
    eval($maxTroop != null)
then
    $result.setResult("You should cast " + $maxSpell.getName() + " spell on " + $maxTroop.getName() + " because it's a ranged troop.");
    update($result);
end

```

Слика 5.10 - Бацање магије на *ranged* јединице

На крају, уколико не постоје трупе осетљиве на магију, нити трупе које су типа *ranged*, систем као последњу опцију бира трупе са највећим здрављем као циљ напада. Пример ове логике се може видети на слици [Слика 5.11](#), где правило “*Cast spell on unit with most health*” идентификује непријатељску јединицу која има највише укупних поена здравља. У првом кораку одређује се магија која има највећу јачину напада, а затим се помоћу функције *accumulate* израчунава која непријатељска трупа има највећи збир поена здравља на основу броја јединица и њиховог индивидуалног здравља. Када се пронађе та трупа, систем препоручује да се најјача магија баци на њу.

```

rule "Cast spell on unit with most health"
agenda-group "spell-casting"
salience 3
when
    $result : ResultRules()
    $enemyUserTroopRule : UserTroopRule(getIsFriendly() == false)
    $spells : List() from collect(Spell())

    $maxSpellDamage : Number() from accumulate(
        Spell( $damage : damage ),
        max($damage)
    )
    $maxSpell : Spell( damage == $maxSpellDamage )

    $maxHealth : Number() from accumulate(
        TroopRule( $health : (troopHealthPoints * (troopCount - 1) + totalHealthPoints) ) from $enemyUserTroopRule.getTroops(),
        max($health)
    )

    $maxTroop : TroopRule( (troopHealthPoints * (troopCount - 1) + totalHealthPoints) == $maxHealth ) from $enemyUserTroopRule.getTroops()
then
    $result.setResult("You should cast " + $maxSpell.getName() + " spell on " + $maxTroop.getName() + " because it has the most health.");
    update($result);
end

```

Слика 5.11 - Бацање магије на најјаче јединице

Backward chaining је приступ у систему правила који се користи за доношење закључака кроз ретроактивну анализу, односно почевши од жељеног исхода и тражећи чињенице и правила која воде до тог исхода. Систем почиње са хипотезом коју жели да потврди и рекурзивно проверава која правила и подаци су потребни да би та хипотеза била истинита. *Backward chaining* је користан када је циљ јасно дефинисан, а потребно је пронаћи или потврдити услове који воде до тог циља, омогућавајући ефикасан и структурисан приступ решавању проблема.

Уколико није могуће бацити магију, јединице типа *melee* примењују сет правила која одређују њихово понашање у борби, водећи рачуна о позиционирању и избору циља. За ово систем користи *backward chaining*, јер јединице започињу од жељеног исхода, и рекурзивно проверавају која правила су потребна да би тај исход био остварен. Правила су организована у три групе:

- Напад на непријатеља у близини пријатељских *ranged* јединица како би их заштитили
- Напад на друге непријатељске јединице уколико претходни услов није испуњен
- Кретање јединице уколико не постоје непријатељске трупе довољно близу.

За проверу да ли постоји непријатељска трупа у близини и одређивање могућих корака које *Melee* јединица може предузети, систем користи структуру података типа дрво (*tree*), које симулира мрежу могућих позиција које јединица може достићи у оквиру свог домета кретања. На основу почетне позиције активне јединице и њене брзине кретања, генерише се дрво *Node* објекта које представља све достижне позиције. Сваки *Node* бележи координате, као и своју везу са родитељским *Node*-ом, чиме се омогућава праћење путање кроз које јединица може стићи до циља.

Након креирања дрвета, сваки *Node* се убацује у *Drools* сесију *bwBase* као објекат *NodeRule*. На тај начин, правило може да провери да ли било која непријатељска трупа заузима позицију која је доступна унутар дрвета кретања активне јединице. Уколико се утврди да је непријатељ присутан у дрвету, правило "*Check If Enemy In Tree*" се активира и означава да јединица може да изврши напад, док се сесија *Drools*-а затим зауставља. Овај приступ омогућава ефикасно одређивање циљева и планирање оптималног кретања, без потребе за ручним прорачуном свих могућих путања. Логика и синтакса која се користи за ову проверу приказана је на слици [Слика 5.12](#).

```
query isContainedIn(int $value1, int $value2, int $i, int $j)
    NodeRule($value1, $value2, $i, $j;)
or
    (NodeRule(z, w, $i, $j;) and isContainedIn($value1, $value2, z, w;))
end

rule "Check If Enemy In Tree"
when
    $troopNode : Tile()
    $i : Integer() from $troopNode.getI()
    $j : Integer() from $troopNode.getJ()
    isContainedIn($i, $j, parameter1, parameter2;)
then
    System.out.println("Enemy troop found in the tree");
    drools.halt();
end
```

Слика 5.12 - Шаблон за процену преклапања поља

У систему правила, *template* представља начин за дефинисање динамичких и поновљивих образаца правила који се могу компајлирати из *spreadsheet*-ова или других извора података. Они омогућавају да се један образац правила примени на више конкретних ситуација тако што се специфични параметри убацују у *template*, што значајно поједностављује управљање комплексним логикама и смањује потребу за ручним писањем појединачних правила. *Template*-и су корисни када се правила заснивају на подацима који се често мењају или када је потребно генерисати правила за различите категорије, прагове или типове јединица без потребе за дуплирањем кода.

У приказаном примеру на слици [Слика 5.13](#), *template*-и се користе за рангирање и класификацију трупа на основу њихове борбене снаге [36]. Подаци као што су *percentThreshold*, *isAdvantage* и *category* се читавају из *spreadsheet*-а, након чега систем аутоматски генерише конкретна правила која се потом извршавају у *Drools* окружењу. Логика *template*-а упоређује збирну снагу пријатељских и непријатељских трупа и на основу процента предности одређује категорију битке. Ако је предност већа од дефинисаног прага, битка се класификује као повољна или неповољна у зависности од вредности параметра *isAdvantage*.

```

rule "Categorize Battle_@row.rowNumber"
when
    $result : ResultRules()
    $userTroopRule : UserTroopRule(getIsFriendly() == true)
    $ourStrength : Number() from accumulate (
        $troop : TroopRule() from $userTroopRule.getTroops(),
        sum($troop.getFightValue() * $troop.getTroopCount())
    )
    $userTroopRule2 : UserTroopRule(getIsFriendly() == false)
    $enemyStrength : Number() from accumulate (
        $troop : TroopRule() from $userTroopRule2.getTroops(),
        sum($troop.getFightValue() * $troop.getTroopCount())
    )
    eval(
        ((@isAdvantage) == 1) && ($ourStrength.doubleValue() > $enemyStrength.doubleValue()) && ($ourStrength.doubleValue() / $enemyStrength.doubleValue() - 1) * 100 > @percentThreshold))
    || ((@isAdvantage) == 0) && ($enemyStrength.doubleValue() > $ourStrength.doubleValue()) && ($enemyStrength.doubleValue() / $ourStrength.doubleValue() - 1) * 100 > @percentThreshold))
)
then
    $result.setResult("Battle classified as: @category}. Our troops have " + $ourStrength.doubleValue() + " strength, enemy troops have " + $enemyStrength.doubleValue() + " strength.");
end
end template

```

Слика 5.13 – Template за процену јачине трупа

Унутар *pom.xml* фајла дефинисане су зависности које омогућавају исправан рад *Kjar* пројекта и његову интеграцију са осталим слојевима система. Најважније међу њима су:

- *spring-boot-starter* – основна зависност која обезбеђује инфраструктуру *Spring Boot* апликације и омогућава једноставно конфигурисање и покретање *Kjar* модула.
- *kie-ci*, *kie-api* – представљају кључне библиотеке *KIE (Knowledge Is Everything)* система, неопходне за управљање базама знања и омогућавање извршавања правила у *Drools* окружењу.
- *drools-core*, *drools-compiler*, *drools-decisiontables* – централни делови *Drools* механизма. Док *drools-core* обезбеђује саму основу за извршавање правила, *drools-compiler* омогућава компајлирање *DRL* фајлова, а *drools-decisiontables* подршку за рад са табелама одлука
- *poi*, *poi-ooxml* – библиотеке за рад са *Microsoft Office* документима, пре свега *Excel* табелама, што је значајно уколико се правила или подаци дефинишу преко табела одлука у *XLS* формату.

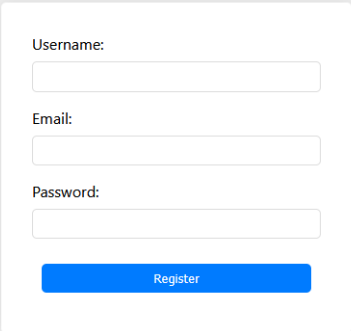
6. Приказ имплементираног система

У претходним поглављима фокус је био на архитектури и унутрашњем функционисању система, док ће у овом поглављу бити приказано имплементирано решење из угла корисника у систему. Циљ поглавља јесте да се прикаже како се теоријски концепти претварају у практичну примену, односно како систем изгледа и понаша се у реалном коришћењу.

6.1. Register страница

Register страница представља основни интерфејс за креирање новог корисничког налога у апликацији, и омогућава кориснику да добије приступ свим функционалностима система. Уколико корисник нема претходно направљен налог, неопходно је да се навигира на ову страницу како би започео регистрацију. На самој страници су приказана поља за унос основних података, који укључују мејл, корисничко име и шифру. Уколико су сви подаци валидни и да не постоји постојећи налог са истим мејлом или корисничким именом, систем креира кориснички налог. У супротном, приказује се порука о грешци и омогућава кориснику да исправи неважеће податке.

Изглед *Register* странице се може видети на слици [Слика 6.1](#).



The image shows a web form titled "Register" centered on a light gray background. The form is a white rectangle with a thin gray border. It contains three input fields: "Username:", "Email:", and "Password:", each followed by a white rectangular input box. Below the input boxes is a blue button with the text "Register" in white. At the bottom of the form, there is a link that says "Already have an account? [Login](#)".

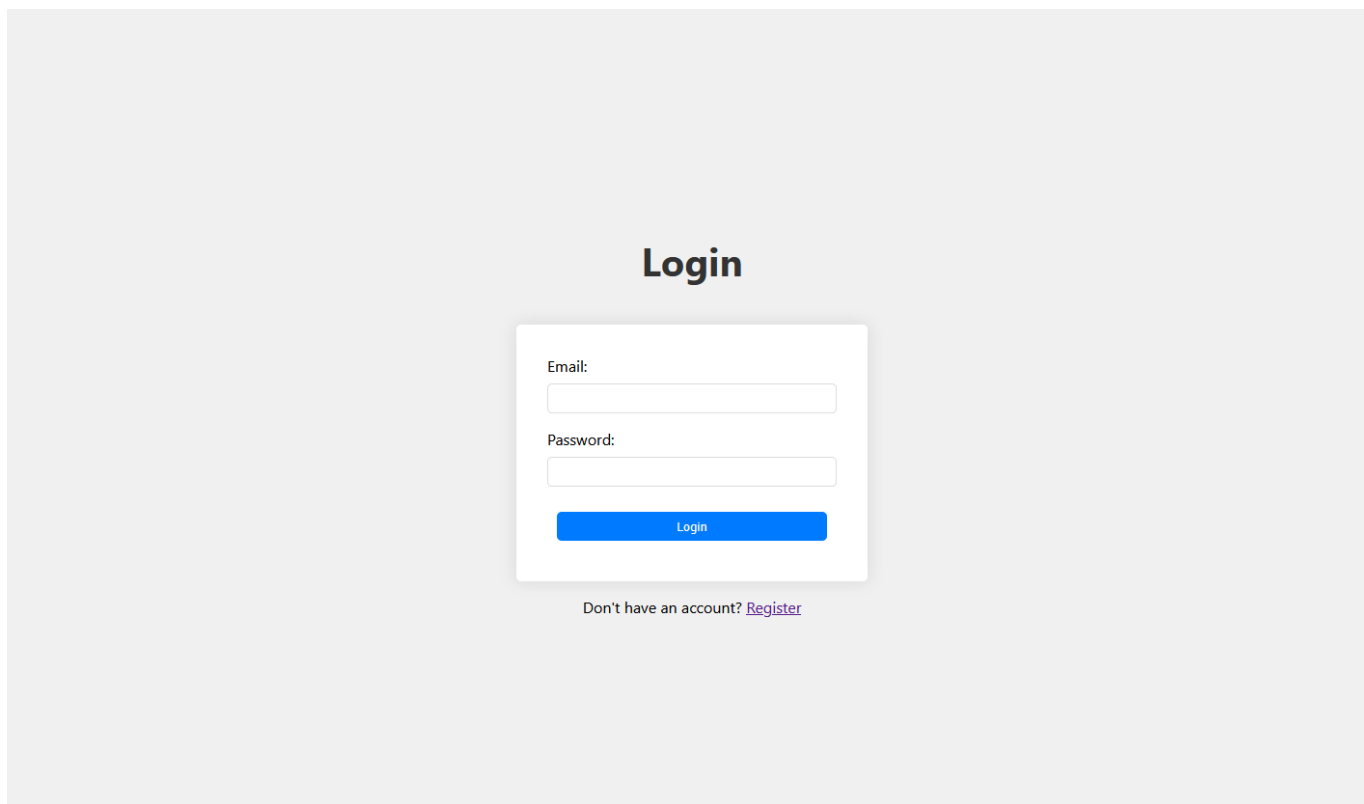
Слика 6.1 - Register страница

6.2. Login страница

Login страница представља интерфејс за аутентификацију корисника који већ поседују налог у систему и омогућава приступ свим функционалностима апликације. Уколико корисник већ има креиран налог, неопходно је да унесе своје корисничко име и шифру у предвиђена поља. Након уноса података, иницира се провера валидности информација од стране система. У случају да су унети подаци тачни и одговарају постојећем налогу, кориснику се омогућава приступ апликацији. У супротном, приказује се порука о грешци

која указује на неправилност у уносу и омогућава кориснику да поново унесе исправне податке. По успешној пријави, корисник се преусмерује на почетни интерфејс апликације, чиме добија приступ свим доступним функцијама система.

Изглед *Login* странице се може видети на слици [Слика 6.2](#).



Слика 6.2 - Login страница

6.3. Strategy страница

Strategy страница представља централни интерфејс апликације, намењен симулацији и управљању борбеним сценаријима. За разлику од претходних страница које су служиле искључиво за аутентификацију и креирање налога, овде корисник добија могућност да активно учествује у стварању и тестирању различитих сценарија. Страница је подељена на два главна дела – *Troop Selection* и *Magic Selection*.

Troop Selection део омогућава кориснику да дода и конфигурише пријатељске и непријатељске трупе, како би се симулирала реална борбена ситуација. Приликом уноса, корисник може да дефинише тип трупе, њену позицију на бојном пољу, количину јединица, број преосталих живота, као и додатне атрибуте који утичу на ток игре, попут информације да ли је јединица већ чекала у овом потезу или да ли је тренутно на потезу.

Изглед *Troop Selection* се може видети на слици [Слика 6.3](#).

Add Our Troop
Add Enemy Troop

Our Troops

Troop Name:

Count:

Health:

i Pos:

j Pos:

☐ Has Waited
 ☒ On Move

Remove

Troop Name:

Count:

Health:

i Pos:

j Pos:

☒ Has Waited
 ☐ On Move

Remove

Troop Name:

Count:

Health:

i Pos:

j Pos:

☒ Has Waited
 ☐ On Move

Remove

Enemy Troops

Troop Name:

Count:

Health:

i Pos:

j Pos:

Remove

Troop Name:

Count:

Health:

i Pos:

j Pos:

Remove

Слика 6.3 - Селекција трупа

Magic Selection део представља интерфејс у коме корисник управља употребом магија у оквиру игре. У оквиру *Magic Selection* селекције, корисник има могућност да одабере које магије поседује, као и услове под којима их користи. На интерфејсу су доступне опције за избор типа терена, број *spell* поена који стоје на располагању, као и ознака да ли је магија већ бачена у текућем потезу. Циљ овог дела апликације јесте да кориснику пружи могућност избора и управљања над сопственим магијским ресурсима.

Изглед *Magic Selection* се може видети на слици [Слика 6.4](#).

Select available spells:

Select terrain:

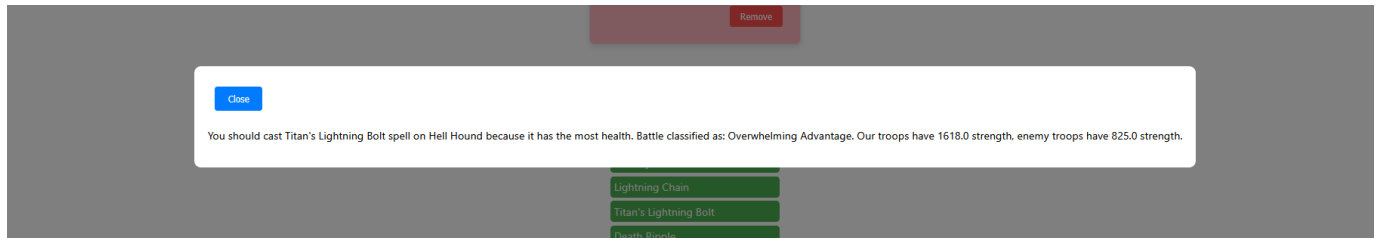
Spell Points:

Used Spell:

☒

Слика 6.4 - Селекција магија

Након што корисник унесе све релевантне податке о трупама и магијама, систем покреће процес обраде правила и доноси одлуку о најбољем потезу у датом тренутку. Коначни резултат се приказује у виду прозора са јасним објашњењем препоручене акције. На примеру са слике [Слика 6.5](#). се види да систем предлаже бацање конкретне магије на одређену непријатељску јединицу, уз навођење разлога за тај избор (нпр. највише животних поена код мете). Поред тога, приказује се и класификација односа снага у битки, као и конкретна вредност снаге пријатељских и непријатељских трупа. На овај начин корисник добија транспарентан увид у препоруку система и лакше може да разуме основу донете одлуке.



Слика 6.5 - Предложена стратегија од стране система

7. Закључак

У овом раду је детаљно представљен развој експертског система за препоруку тактике у игри *Heroes of Might and Magic III*, заснованог на систему правила. Систем се састоји из три дела: серверске апликације имплементиране у *SpringBoot*-у, система правила реализованог у *Drools*-у и клијентске апликације која омогућава кориснику да уноси податке о трупима и магијама и да добија препоруке за најбољи потез. Знање које је коришћено за развој система ослоњено је на постојећа правила игре и тактичке принципе који се користе у пракси.

Описана су слична решења, употребљене технологије и спецификација система. На крају рада приказан је начин употребе апликације из перспективе корисника, од регистрације и пријаве, преко избора трупа и магија, до добијања конкретне препоруке за најбољи потез у борби.

Могући правци даљег развоја система обухватају проширивање базе правила новим тактикама, интеграцију сложених услова, као и увођење адаптивних правила која би могла да уче из претходних партија, чиме би се омогућило прилагођавање препорука стилу игре самог корисника. Пред тога, систем би се могао унапредити омогућавањем играчима да сами дефинишу правила или измене већ постојећа правила, чиме би се постигла већа флексибилност и интерактивност. Такође, систем тренутно не урачунава трупе које су имуне на магију, нити узима у обзир специјалне ефекте који могу бити активни на јединицама током борбе, што представља простор за додатно унапређење.

8. Литература

- [1] M. Widomska, „What’s The Deal With Heroes of Might and Magic III?“, 30 July 2020. <https://web.archive.org/web/20201029025340/http://loading.bar/articles/whats-the-deal-with-heroes-of-might-and-magic-iii>.
- [2] Heroes of Might and Magic Wiki, „Strategies“, 26 August 2025. <https://heroes.thelazy.net/index.php/Strategies>.
- [3] D. M. Bourg, „Chapter 11. Rule-Based AI“, y *AI for Game Developers*, O'Reilly Media Inc, 2004, p. 392.
- [4] É. Piette, „Ludii“, 21 February 2020. <https://arxiv.org/abs/1905.05013>.
- [5] D. Yang, „Stockfish“, <https://stockfishchess.org/>. [Последњи приступ 5 October 2025].
- [6] RUFAl, *Review of Strategies in Games Theory and Its Application In*, 2021.
- [7] T. Xenofex, „Heroes III Assist“, 4 September 2025. <https://www.heroes3assist.com/>.
- [8] D. Rudnov, „Heroes III Tools“, 4 September 2025. <https://heroes3tools.netlify.app/>.
- [9] D. Milosavljević, „Heroes of Might And Magic III Battle Calculator“, 4 September 2025. <https://dusanmilosavljevic1624.github.io/homm3-calc/>.
- [10] S. Vladimir, „FreeHeroes engine – Battle Emulator“, 29 April 2025. <https://heroes3wog.net/freeheroes-engine-battle-emulator/>.
- [11] T. Ono, „Heroes of Might and Magic III Official Strategy Guide“, 1999. <https://archive.org/details/homm-3-guide>.
- [12] E. Chin, „GameStop Unofficial Game Guide to Heroes of Might and Magic III“, 1999. https://archive.org/details/Heroes_of_Might_Magic_3_Manual.
- [13] React, „React“, 4 September 2025. <https://react.dev/>.
- [14] Wikipedia, „Spring Boot“, 4 September 2025. https://en.wikipedia.org/wiki/Spring_Boot.
- [15] PostgreSQL, „PostgreSQL“, 4 September 2025. <https://www.postgresql.org/>.
- [16] Drools, „Drools“, 4 September 2025. <https://www.drools.org/>.
- [17] Docker, „Docker“, 4 September 2025. <https://www.docker.com/>.
- [18] Matéu.sh, „What is the Virtual DOM in React?“, 5 June 2024. <https://www.freecodecamp.org/news/what-is-the-virtual-dom-in-react/>.
- [19] W3Schools, „React Lifecycle“, 4 September 2025. https://www.w3schools.com/react/react_lifecycle.asp.
- [20] GeeksForGeeks, „ReactJS State vs Props“, 4 September 2025. <https://www.geeksforgeeks.org/reactjs/reactjs-state-vs-props/>.
- [21] Baeldung, „<https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring>“, 4 September 2025. <https://www.baeldung.com/inversion-control-and-dependency-injection-in-spring>.
- [22] Scaler Academy, „Spring Boot Architecture – Detailed Explanation“, 16 August 2023. <https://www.interviewbit.com/blog/spring-boot-architecture/>.

- [23] Apache Maven Project, „Introduction to the Dependency Mechanism,“ 31 August 2025. <https://maven.apache.org/guides/introduction/introduction-to-dependency-mechanism.html>.
- [24] Spring, „Spring Initializr,“ 4 September 2025. <https://start.spring.io/>.
- [25] jboss.org, „User Guide,“ 4 September 2025. <https://docs.drools.org/5.2.0.M2/drools-expert-docs/html/ch04.html>.
- [26] jboss.org, „The Rule Language,“ 4 September 2025. [Na mreži]. Available: <https://docs.jboss.org/drools/release/5.2.0.Final/drools-expert-docs/html/ch05.html>.
- [27] Baeldung, „An Example of Backward Chaining in Drools,“ 4 September 2025. <https://www.baeldung.com/drools-backward-chaining>.
- [28] GeeksForGeeks, „Use Case Diagram - Unified Modeling Language,“ 23 July 2025. <https://www.geeksforgeeks.org/system-design/use-case-diagram/>.
- [29] GeeksForGeeks, „UML Class Diagram,“ 29 August 2025. <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-class-diagrams/>.
- [30] GeeksForGeeks, „Sequence Diagrams - Unified Modeling Language (UML),“ 03 January 2025. <https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-sequence-diagrams/>.
- [31] JWT, „JSON Web Token,“ 4 September 2025. <https://www.jwt.io/>.
- [32] Spring, „Creating a Multi Module Project,“ 4 September 2025. <https://spring.io/guides/gs/multi-module>.
- [33] Pankaj Kumar, „JPA Annotations - Hibernate Annotations,“ 4 August 2022. <https://www.digitalocean.com/community/tutorials/jpa-hibernate-annotations>.
- [34] Spring, „Spring Security,“ 4 September 2025. <https://spring.io/projects/spring-security>.
- [35] Heroes of Might and Magic Wiki, „Vulnerability,“ 4 September 2025. <https://heroes.thelazy.net/index.php/Vulnerability>.
- [36] Heroes of Might and Magic Wiki, „Fight Value,“ 2 September 2025. https://heroes.thelazy.net/index.php/Fight_Value.

9. Биографија

Немања Шимшић је рођен 02. фебруара 2002. године у Зрењанину. Завршио је основну школу „Петар Петровић Његош“, и електротехничку и грађевинску школу „Никола Тесла“. Године 2020. уписао је Факултет техничких наука у Новом Саду, смер Софтверско инжењерство и информационе технологије. Полаже све испите предвиђене планом и програмом и основне академске студије завршава 2025. године.